Отчёт о воспроизведении экспериментов Exclusive Self Attention

1. Реализация и экспериментальная установка

Для частичного воспроизведения результатов статьи “Exclusive Self Attention” я самостоятельно реализовал на PyTorch авторегрессионную языковую модель MiniGPT с поддержкой обычного Self Attention и Exclusive Self Attention.

Также самостоятельно реализовал и обучил BPE токенизатор со словарём из 4096 токенов. Обучение токенизатора проводилось на фрагменте WikiText-103 объёмом около 5 МБ. Для обучения языковой модели использовался WikiText-103.

Проверялись две конфигурации Transformer:

Параметр	**Small**	**Medium**
Количество параметров	13,79 млн	29,41 млн
Количество слоёв	6	8
Размер эмбеддингов	384	512
Количество голов	6	8

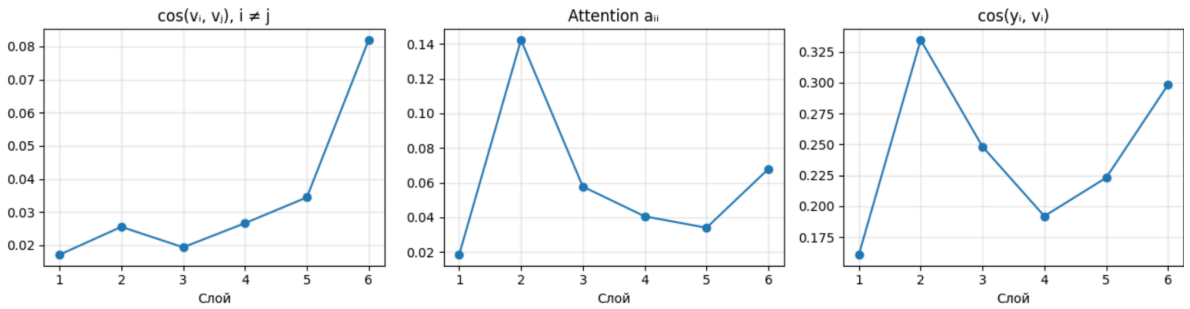
В моделях использовались RoPE, LayerNorm, AdamW, linear warmup и cosine decay. Каждая конфигурация обучалась 1000 шагов с gradient accumulation = 4.

2. Воспроизведение Figure 1: Attention Similarity Bias

Я исследовал обученную Small-модель со стандартным SA, измеряя по слоям:

- Среднее косинусное сходство value-векторов разных позиций.
- Средний вес внимания токена к самому себе.
- Косинусное сходство выхода Attention и value-вектора текущего токена.

Получено положительное сходство между выходом Attention и собственным value-вектором: приблизительно от 0,16 до 0,33 в зависимости от слоя.

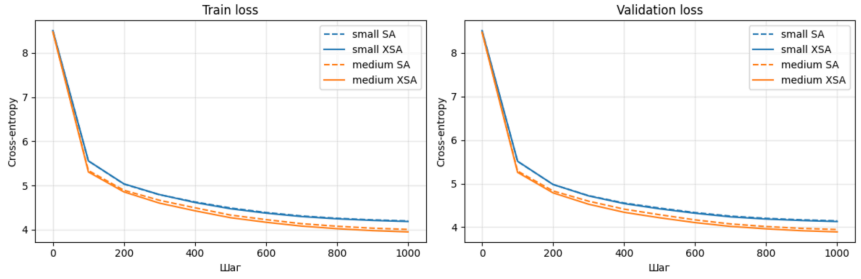


Таким образом, наблюдение авторов об attention similarity bias подтвердилось.

3. Воспроизведение Figures 3–4: размер модели и learning rate

Я сравнил SA и XSA на двух размерах модели при learning rate 3e-4 и 6e-4.

Train и Validation loss:

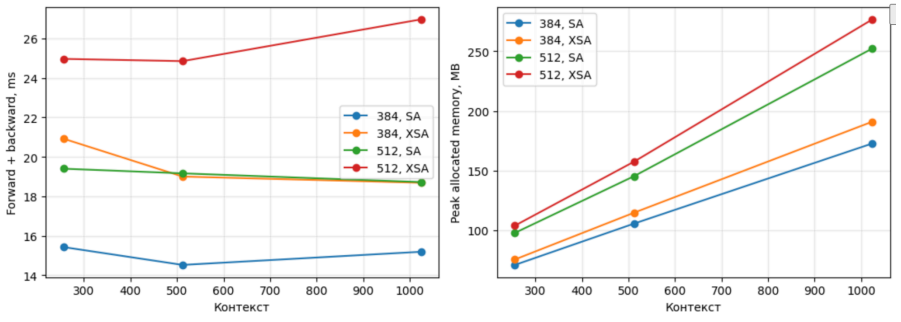


XSA показал более низкий loss во всех конфигурациях.

При learning rate 6e-4 преимущество XSA составило 0,0154 для Small и 0,0541 для Medium. Таким образом, в моих экспериментах преимущество XSA оказалось больше на более крупной модели, что согласуется с результатом статьи.

4. Воспроизведение Figure 2: вычислительные затраты

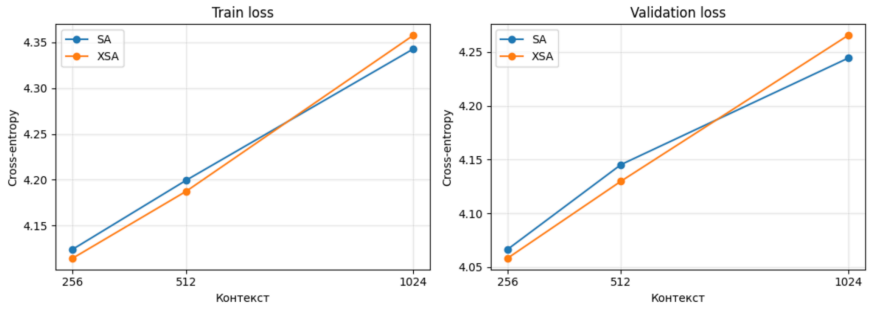
Я измерил время forward + backward и пиковое потребление памяти для одного Transformer блока с SA и XSA.



Таким образом, XSA имеет измеримые вычислительные дополнительные расходы. В отличие от результатов авторов, на моей реализации относительное увеличение времени оказалось более большим, что связано с размером модели.

5. Воспроизведение Figure 5: длина контекста

Я сравнил Small модели при длинах контекста 256, 512 и 1024 токена. Количество обрабатываемых токенов на шаг сохранялось одинаковым за счёт изменения batch size.

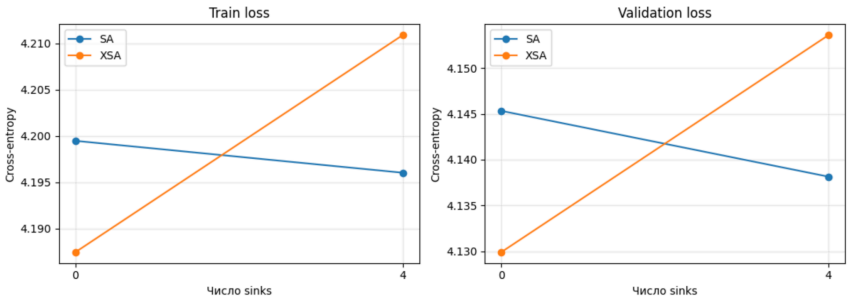


При контексте 256 и 512 XSA показал преимущество. Однако при 1024 токенах результат XSA оказался хуже SA.

Следовательно, увеличение преимущества XSA на длинных контекстах, наблюдаемое в оригинальной статье, воспроизвести не удалось.

6. Воспроизведение Figure 6: Attention Sinks

Я дополнительно исследовал влияние четырёх обучаемых префиксных sink токенов на качество моделей.



Добавление sink токенов немного улучшило результат SA, но ухудшило результат XSA.

В данной экспериментальной постановке совместное использование XSA и обучаемых sink токенов не дало преимущества.

7. Вывод

При более маленькой модели удалось подтвердить основной результат статьи: XSA может улучшать качество языкового моделирования относительно стандартного Self Attention без увеличения числа обучаемых параметров.

Преимущество наблюдалось для обеих исследованных архитектур и двух значений learning rate. Также удалось частично воспроизвести явление attention similarity bias.

Однако улучшение XSA на более длинных контекстах и при использовании дополнительных attention sinks не воспроизвелось. Кроме того, XSA увеличивал время вычислений и потребление памяти.

Ограничениями экспериментов являются небольшой масштаб моделей, короткое обучение. Поэтому результаты демонстрируют работоспособность метода в выбранных условиях.